

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ – VIỄN THÔNG



HCMUS
Viet Nam National University
Ho Chi Minh City
University of Science

BÁO CÁO ĐỒ ÁN
MÔN: THỰC HÀNH GIAO TIẾP MÁY TÍNH VÀ THU NHẬN DỮ LIỆU
TÊN ĐỀ TÀI: HỆ THỐNG ĐO NHIỆT ĐỘ VÀ ĐỘ ẨM
Lớp: 22DTV_CLC1

Giảng viên hướng dẫn:

- ThS. Đặng Tấn Phát
- CN. Hồ Thanh Bảo

Sinh viên thực hiện:

Họ và tên: Lương Hải Long
Mã số sinh viên: 22207056

THÀNH PHỐ HỒ CHÍ MINH - THÁNG 12 NĂM 2025

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ – VIỄN THÔNG

BÁO CÁO ĐỒ ÁN
MÔN: THỰC HÀNH GIAO TIẾP MÁY TÍNH VÀ THU NHẬN DỮ LIỆU
TÊN ĐỀ TÀI: HỆ THỐNG ĐO NHIỆT ĐỘ VÀ ĐỘ ẨM
Lớp: 22DTV_CLC1

Giảng viên hướng dẫn:

- ThS. Đặng Tấn Phát
- CN. Hồ Thanh Bảo

Sinh viên thực hiện:

Họ và tên: Lương Hải Long
Mã số sinh viên: 22207056

PHIẾU ĐÁNH GIÁ ĐỒ ÁN
THỰC HÀNH GIAO TIẾP MÁY TÍNH VÀ THU NHẬN DỮ LIỆU

I. Phần bắt buộc:

Tiêu chí	Điểm	Tổng điểm tối đa	Đánh giá
Demo hoạt động tốt	1		
Mức độ hoàn thiện và độ tin cậy của dữ liệu	1		
Giải thích nguyên lý hoạt động	1		
Sử dụng ngắt, timer, uart, adc, ...	0,5		
Sử dụng cơ sở dữ liệu	0,5		
Cuốn báo cáo trình bày đầy đủ nội dung	0.5		
Slide trình bày dễ hiểu	0.5		

II. Phần điểm cộng:

Tiêu chí	Điểm	Tổng điểm tối đa	Đánh giá
Demo hoạt động tốt	0,5		
Giải thích nguyên lý và thuật toán	0,5		
Mức độ hoàn thiện và độ tin cậy của dữ liệu	1		

STT Nhóm:

Điểm tổng kết:

....., ngày ... tháng ... năm 2025

Người đại diện nhóm
(Ký tên, ghi rõ họ tên)

Lương Hải Long

BẢNG PHÂN CHIA CÔNG VIỆC

HỌ TÊN	TIẾN ĐỘ	CÔNG VIỆC	MÔ TẢ CÔNG VIỆC
LƯƠNG HẢI LONG	100%	Soạn báo cáo	Soạn và viết tài liệu báo cáo bằng Latex
		Soạn slide	Soạn và tóm tắt nội dung vào slide bằng PowerPoint
		Phát triển	Lập trình Firmware EFR32 (I2C, BLE, UART).
		Lắp ráp	Lắp ráp cảm biến và bo mạch
		Viết App	Viết App PC (Python + SQLite).
		Kiểm tra chất lượng	Kiểm tra lại toàn bộ nội dung của nhóm
		Thuyết trình	Phân tích và trình bày nội dung đồ án của nhóm

MỤC LỤC

Trang bìa	1
Trang phụ bìa	2
Phiếu đánh giá đồ án	3
Bảng phân chia công việc	4
Mục lục	5
Câu 1: Ứng dụng hệ thống đo nhiệt độ và độ ẩm	7
1. Tổng quan hệ thống	7
2. Kiến trúc hệ thống phần cứng và phần mềm	7
2.1. Sơ đồ khối hệ thống	7
2.2. Cơ chế hoạt động của Firmware	7
3. Phân tích quy trình hoạt động và giải thuật	7
3.1. Giao tiếp cảm biến AHT20 qua I2C	7
3.2. Giải thuật xử lý số liệu và Tiết kiệm tài nguyên	8
3.3. Giao thức truyền thông không dây BLE	8
3.4. Ứng dụng PC và Cơ sở dữ liệu SQLite	9
4. Kết quả thực nghiệm	9
4.1. Hoạt động của Phần cứng (Firmware)	9
4.2. Hoạt động của Ứng dụng PC và Truyền tin	10
4.3. Cơ sở dữ liệu và Tính toán vận dữ liệu	11
5. Kết luận và Hướng phát triển	11
5.1. Kết luận	11
5.2. Hướng phát triển	11
Câu 2: Phát triển hệ thống định vị trong nhà (Indoor Positioning)	13
1. Tổng quan vấn đề	13
2. Kiến trúc hệ thống và Mô hình không gian	13
2.1. Cấu hình phần cứng (Beacons)	13
2.2. Phân chia khu vực (Rooms)	13
3. Giải thuật và Quy trình xử lý trên Mobile App	13
3.1. Quét và Lọc thiết bị (Scanning & Filtering)	13
3.2. Giải mã dữ liệu cảm biến (Data Parsing)	13
3.3. Ước lượng khoảng cách từ RSSI	13
3.4. Giải thuật Đo ba cạnh (Trilateration)	14
4. Kết quả thực nghiệm	14
4.1. Giao diện người dùng	14
4.2. Đánh giá độ chính xác	15
5. Kết luận	16
Phụ lục: Mã nguồn quan trọng	17
A. Firmware EFR32 (C Language)	17
B. PC Application (C/Win32 API)	21
C. Android Application (Kotlin)	22

DANH MỤC HÌNH ẢNH

Hình 1	Kết quả hiển thị thông số môi trường trên màn hình LCD	10
Hình 2	Giao diện phần mềm thu thập dữ liệu qua UART trên máy tính	10
Hình 3	Dữ liệu lịch sử đo đạc được lưu trữ trong SQLite	11
Hình 4	Giao diện định vị thời gian thực trên Android	15
Hình 5	Dữ liệu chi tiết thu thập được từ các Anchor Nodes	16

Câu 1: Ứng dụng hệ thống đo nhiệt độ và độ ẩm

1. Tổng quan hệ thống

Trong bối cảnh IoT (Internet of Things) phát triển mạnh mẽ, yêu cầu thu thập dữ liệu môi trường chính xác và truyền tải đa nền tảng là bài toán cốt lõi. Đồ án này tập trung giải quyết vấn đề thu thập dữ liệu nhiệt độ, độ ẩm từ cảm biến chuyên dụng, hiển thị tại chỗ, đồng thời truyền tải dữ liệu qua cả hai kênh: có dây (UART) để lưu trữ lâu dài và không dây (BLE) để giám sát di động.

Mục tiêu cụ thể:

- Xây dựng driver giao tiếp mức thấp (Low-level) cho cảm biến AHT20.
- Thiết kế hệ thống nhúng trên vi điều khiển EFR32xG21 hoạt động theo cơ chế hướng sự kiện (Event-driven).
- Lập trình ứng dụng PC bằng ngôn ngữ C (Win32 API) kết hợp cơ sở dữ liệu SQLite để lưu trữ lịch sử đo đạc.

2. Kiến trúc hệ thống phần cứng và phần mềm

2.1. Sơ đồ khối hệ thống

Hệ thống bao gồm 3 khối chính:

- Khối Thu thập (Sensor Node):** Cảm biến AHT20 kết nối với Kit EFR32 qua giao tiếp I2C.
- Khối Xử lý trung tâm (MCU):** Vi điều khiển EFR32BG22 thực hiện đọc cảm biến, hiển thị LCD, đóng gói gói tin BLE và gửi log UART.
- Khối Lưu trữ & Giám sát (Host/Gateway):** Máy tính chạy phần mềm App.exe (C Language) để nhận dữ liệu qua cổng COM và ghi vào file `sensor_data.db`.

2.2. Cơ chế hoạt động của Firmware

Thay vì sử dụng vòng lặp `while(1)` truyền thống gây lãng phí năng lượng, hệ thống được thiết kế dựa trên ngăn xếp (stack) của Silicon Labs:

- System Boot Event:** Khởi tạo ngoại vi, thiết lập Timer, bắt đầu quảng bá BLE.
- Soft Timer Event:** Sử dụng `app_timer` để tạo ngắt định kỳ (1000ms), kích hoạt quá trình đọc cảm biến mà không chặn (non-blocking) các tác vụ khác của hệ thống.
- BLE Stack Event:** Xử lý các sự kiện liên quan đến kết nối và quảng bá.

3. Phân tích quy trình hoạt động và giải thuật

3.1. Giao tiếp cảm biến AHT20 qua I2C

Nhóm thực hiện viết driver thủ công dựa trên Datasheet AHT20 để đảm bảo kiểm soát hoàn toàn timing (thời gian thực).

Quy trình máy trạng thái (State Machine):

- Khởi tạo (`aht20_init`):** Gửi lệnh `0xBE` để kiểm tra và thiết lập thanh ghi trạng thái, sau đó chờ 10ms để cảm biến ổn định.
- Kích hoạt đo (`Trigger Measurement`):**
 - Gửi chuỗi lệnh: `0xAC` (Trigger), `0x33` (Data 1), `0x00` (Data 2).
 - Wait State:** Vi điều khiển thực hiện `sl_sleeptimer_delay_millisecond(80)`. Đây là khoảng thời gian bắt buộc để bộ ADC bên trong cảm biến chuyển đổi tín hiệu vật lý sang tín hiệu số.

3. Đọc dữ liệu (Read Data): Đọc về 7 bytes từ địa chỉ I2C.

- Byte 0: Status (Trạng thái).
- Byte 1, 2, 3: Chứa dữ liệu độ ẩm (20-bit).
- Byte 3, 4, 5: Chứa dữ liệu nhiệt độ (20-bit).

4. Giải thuật tách bit (Bit Shifting):

- Độ ẩm (raw_hum) được ghép từ byte 1 (dịch trái 12), byte 2 (dịch trái 4) và 4 bit cao của byte 3.
- Nhiệt độ (raw_temp) được ghép từ 4 bit thấp của byte 3, toàn bộ byte 4 và byte 5.
- Công thức chuyển đổi vật lý:

$$T(^{\circ}C) = \left(\frac{\text{raw_temp}}{2^{20}} \right) \times 200 - 50$$

$$RH(\%) = \left(\frac{\text{raw_hum}}{2^{20}} \right) \times 100$$

3.2. Giải thuật xử lý số liệu và Tiết kiệm tài nguyên

Trên các hệ thống nhúng, việc sử dụng thư viện printf với định dạng số thực %f tiêu tốn rất nhiều bộ nhớ Flash và RAM. Nhóm đã tối ưu hóa bằng thuật toán số nguyên (Integer Arithmetic).

Phương pháp “Fixed-point” giả lập:

- Khi cần gửi giá trị $25.5^{\circ}C$ qua UART, thay vì gửi số thực, hệ thống nhân giá trị này với 10 để được số nguyên 255.
- Tách phần nguyên và phần thập phân:

```
1 int t_int = temp10 / 10;    // 255 / 10 = 25
2 int t_frac = temp10 % 10;  // 255 % 10 = 5
```

- Gửi chuỗi ký tự đã định dạng sẵn: `app_log("D:%d.%d...", t_int, t_frac)`. Điều này giúp giảm đáng kể kích thước mã nguồn (Code size).

3.3. Giao thức truyền thông không dây BLE

Để các thiết bị di động có thể nhận dữ liệu mà không cần ghép nối (Pairing), hệ thống sử dụng cơ chế **BLE Advertising** (Quảng bá).

Cấu trúc gói tin tối ưu (BCD Encoding): Dữ liệu nhiệt độ/độ ẩm được nén vào trường Manufacturer Specific Data của gói tin quảng bá. Thay vì dùng mã ASCII (tốn 1 byte cho mỗi ký tự số), nhóm sử dụng mã BCD (Binary Coded Decimal):

- **Giải thuật nén:** 2 chữ số thập phân được đóng gói vào 1 byte.
 - Công thức: `packed_byte = (hang_chuc << 4) | hang_don_vi`.
 - Ví dụ: Giá trị 25 -> 0x25 (nhị phân 0010 0101).
- **Cấu trúc Payload (31 bytes):**
 - Flags: 0x02, 0x01, 0x06 (General Discoverable).
 - Local Name: "F3".
 - Company ID: 0x02FF.

• Data: 4 bytes [T_Int][T_Dec][H_Int][H_Dec] đã nén BCD.

- **Chu kỳ quảng bá:** Được thiết lập 50ms (80 x 0.625ms) thông qua hàm `sl_bt_advertiser_set_timing`, đảm bảo tính cập nhật nhanh cho các ứng dụng định vị hoặc giám sát thời gian thực.

3.4. Ứng dụng PC và Cơ sở dữ liệu SQLite

Phần mềm phía máy tính được xây dựng bằng ngôn ngữ C thuần túy, sử dụng **Win32 API** để giao tiếp phần cứng, chứng minh khả năng lập trình hệ thống mức thấp của nhóm.

1. Giao tiếp Serial (COM Port):

- Sử dụng `CreateFileA` để mở cổng COM.
- Cấu hình DCB (Device Control Block) với Baudrate 115200, 8 data bits, No parity, 1 stop bit.
- Sử dụng `ReadFile` trong vòng lặp vô hạn để lắng nghe dữ liệu đến.

2. Parsing dữ liệu (Phân tích cú pháp):

- Bộ đệm nhận dữ liệu (Buffer) được quét để tìm ký tự xuống dòng `\n`.
- Sử dụng `sscanf` với mẫu `"D:%f,%f"` để khôi phục lại giá trị nhiệt độ và độ ẩm từ chuỗi nhận được.

3. Lưu trữ Cơ sở dữ liệu (SQLite Integration):

- Tích hợp thư viện `sqlite3.c` trực tiếp vào mã nguồn.
- Tự động kiểm tra và tạo bảng `SensorLog` với khóa chính tự tăng (`AUTOINCREMENT`).
- Mỗi khi nhận được gói tin hợp lệ, ứng dụng thực thi câu lệnh SQL `INSERT` kèm theo `Timestamp` (thời gian thực từ máy tính) để đảm bảo tính toàn vẹn dữ liệu.

4. Kết quả thực nghiệm

4.1. Hoạt động của Phần cứng (Firmware)

Qua quá trình kiểm thử liên tục trong thời gian 30 phút, hệ thống trên Kit EFR32xG21 hoạt động ổn định với các kết quả cụ thể sau:

1. **Độ chính xác cảm biến:** Cảm biến AHT20 phản hồi nhanh với các thay đổi môi trường. Dữ liệu đọc được ổn định, không có hiện tượng nhảy giá trị đột ngột (spike) do nhiễu.
2. **Hiển thị trực quan:** Màn hình LCD trên Kit hiển thị đầy đủ và rõ ràng các thông tin theo thiết kế:
 - Dòng 1: Nhiệt độ (Ví dụ: 32.3 C).
 - Dòng 3: Độ ẩm (Ví dụ: 55.3 %).
 - Dòng 5: Chu kỳ lấy mẫu (1000 ms).
 - Dòng 7-8: Thông tin sinh viên (Lương Hải Long, MSSV: 22207056, F3).
3. **Độ ổn định:** Hệ thống không xảy ra hiện tượng treo (Watchdog reset) hay mất kết nối I2C trong quá trình chạy thử nghiệm.

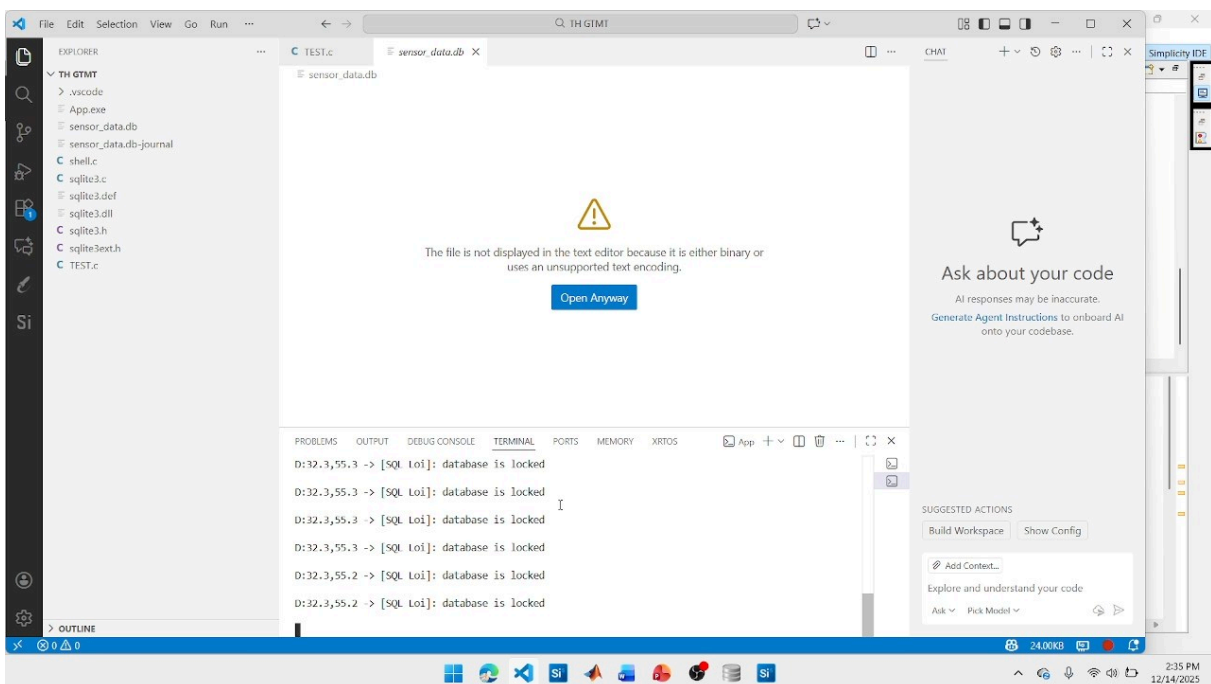


Hình 1: Kết quả hiển thị thông số môi trường trên màn hình LCD

4.2. Hoạt động của Ứng dụng PC và Truyền tin

Ứng dụng App.exe được viết bằng C (Win32 API) đã kết nối thành công với thiết bị qua cổng COM ảo (Ví dụ: COM10) với tốc độ Baud 115200.

1. **Dữ liệu UART:** Console hiển thị dòng log dữ liệu nhận được theo thời gian thực với định dạng D:Temperature, Humidity (Ví dụ: D:32.3,55.4). Tốc độ cập nhật trùng khớp với chu kỳ trên Kit.
2. **Quảng bá BLE:** Sử dụng ứng dụng quét Bluetooth (EFR Connect/nRF Connect) trên điện thoại, thiết bị hiển thị đúng tên “F3”. Dữ liệu Manufacturer Data thay đổi tương ứng với giá trị nhiệt độ và độ ẩm môi trường, chứng tỏ giải thuật nén BCD hoạt động chính xác.

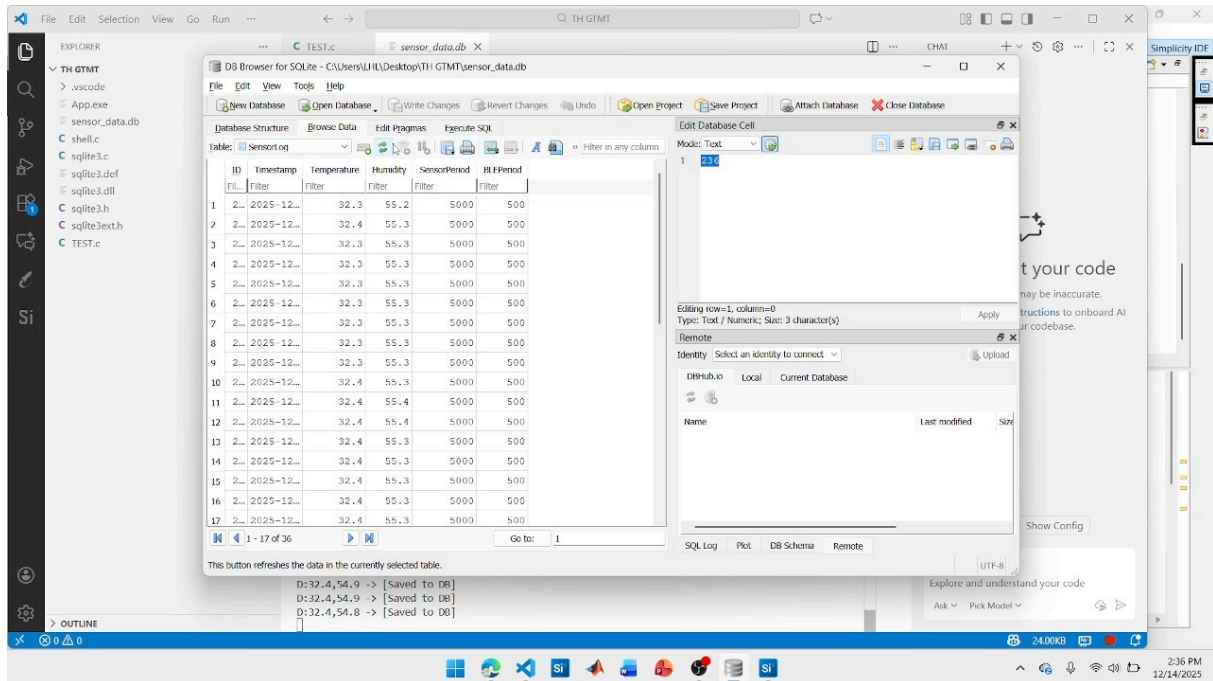


Hình 2: Giao diện phần mềm thu thập dữ liệu qua UART trên máy tính

4.3. Cơ sở dữ liệu và Tính toán vận dữ liệu

Hệ thống lưu trữ hoạt động đúng theo kịch bản đề ra:

1. **Tạo lập Database:** File `sensor_data.db` được tạo tự động ngay khi chạy chương trình.
2. **Kiểm tra dữ liệu:** Sử dụng phần mềm *DB Browser for SQLite* để truy vấn file cơ sở dữ liệu cho thấy:
 - Bảng `SensorLog` chứa đầy đủ các cột: ID, Timestamp, Temperature, Humidity, SensorPeriod.
 - Cột Timestamp ghi nhận chính xác thời gian thực của máy tính tại thời điểm nhận gói tin.
 - Dữ liệu nhiệt độ và độ ẩm trong Database khớp hoàn toàn với dữ liệu hiển thị trên LCD và Console.



Hình 3: Dữ liệu lịch sử đo đạc được lưu trữ trong SQLite

5. Kết luận và Hướng phát triển

5.1. Kết luận

Đồ án (Câu 1) đã hoàn thành trọn vẹn các yêu cầu của đề bài với những điểm nhấn về mặt kỹ thuật:

1. Làm chủ giao thức I2C ở mức thanh ghi (Register Level) để giao tiếp với cảm biến AHT20 mà không phụ thuộc thư viện Arduino.
2. Tối ưu hóa băng thông truyền tin BLE và bộ nhớ UART bằng các giải thuật nén số liệu (BCD Encoding, Integer Math).
3. Xây dựng thành công ứng dụng PC bằng ngôn ngữ C thuần túy, giao tiếp trực tiếp với Hardware và tích hợp Database SQLite, chứng minh khả năng lập trình hệ thống vững chắc.

5.2. Hướng phát triển

Dựa trên kết quả đạt được, hệ thống có thể được nâng cấp theo các hướng sau:

1. **Tối ưu năng lượng:** Sử dụng chế độ ngủ sâu (EM2 Deep Sleep) của vi điều khiển EFR32 giữa các chu kỳ đo để kéo dài thời gian sử dụng pin.
2. **Mở rộng ứng dụng PC:** Vẽ đồ thị (Chart) biến thiên nhiệt độ/độ ẩm theo thời gian thực trực tiếp trên giao diện ứng dụng.

3. **Bảo mật:** Mã hóa gói tin quảng bá BLE để ngăn chặn việc nghe lén dữ liệu môi trường.

Câu 2: Phát triển hệ thống định vị trong nhà (Indoor Positioning)

1. Tổng quan vấn đề

Phát triển từ Câu 1, hệ thống được mở rộng thành một mạng lưới gồm 3 thiết bị BLE (Beacons) đặt tại các vị trí cố định. Người dùng sử dụng thiết bị di động (Smartphone) để thu thập tín hiệu, từ đó xác định tọa độ (x, y) của mình trong thời gian thực và xác định vị trí phòng hiện tại trên bản đồ số.

Mục tiêu kỹ thuật:

- Xây dựng ứng dụng Android (Kotlin) có khả năng quét đồng thời nhiều thiết bị BLE.
- Thực hiện giải thuật định vị **Trilateration** (Đo ba cạnh) dựa trên cường độ tín hiệu RSSI.
- Hiển thị trực quan vị trí người dùng và thông số môi trường (Nhiệt độ, độ ẩm) của từng khu vực.

2. Kiến trúc hệ thống và Mô hình không gian

2.1. Cấu hình phần cứng (Beacons)

Hệ thống sử dụng 3 bộ Kit EFR32xG21 đóng vai trò là các Anchor Node (Điểm neo), được cấu hình với tọa độ cố định trong không gian 8×8 mét:

- Beacon F1:** Tọa độ $(1.0, 1.0)$ mét.
- Beacon F2:** Tọa độ $(7.0, 1.0)$ mét.
- Beacon F3:** Tọa độ $(1.0, 7.0)$ mét.

Mỗi Beacon phát quảng bá gói tin chứa dữ liệu cảm biến và định danh (F1, F2, F3) với tần suất cao ($50-100\text{ms}$) để đảm bảo độ mượt khi định vị.

2.2. Phân chia khu vực (Rooms)

Không gian được chia thành 4 phòng ảo dựa trên tọa độ:

- Phòng 1:** $x \in [0, 4), y \in [0, 4)$.
- Phòng 2:** $x \in [4, 8), y \in [0, 4)$.
- Phòng 3:** $x \in [0, 4), y \in [4, 8)$.
- Phòng 4:** $x \in [4, 8), y \in [4, 8)$.

3. Giải thuật và Quy trình xử lý trên Mobile App

Ứng dụng Android được phát triển bằng ngôn ngữ **Kotlin**, sử dụng thư viện `android.bluetooth.le`. Quy trình xử lý dữ liệu diễn ra theo luồng sau:

3.1. Quét và Lọc thiết bị (Scanning & Filtering)

Ứng dụng sử dụng `BluetoothLeScanner` để quét liên tục. Tại hàm callback `onScanResult`, bộ lọc mềm được áp dụng để chỉ chấp nhận các thiết bị có tên "F1", "F2", "F3".

Dữ liệu thô từ mỗi Beacon được lưu vào danh sách quản lý đối tượng `IndoorBeacon`, bao gồm: RSSI trung bình, thông số môi trường và tọa độ cố định.

3.2. Giải mã dữ liệu cảm biến (Data Parsing)

Tương tự Câu 1, dữ liệu nhiệt độ và độ ẩm được trích xuất từ trường **Manufacturer Specific Data** (ID `0x02FF`) theo chuẩn BCD.

- Byte 0-1: Nhiệt độ (Phần nguyên, Phần thập phân).
- Byte 2-3: Độ ẩm (Phần nguyên, Phần thập phân).
- Công thức tái tạo: $\text{Value} = \text{BcdToInt}(\text{IntPart}) + \text{BcdToInt}(\text{DecPart}) / 10.0$.

3.3. Ước lượng khoảng cách từ RSSI

Khoảng cách d từ điện thoại đến mỗi Beacon được ước lượng dựa trên mô hình suy hao log-normal:

$$RSSI = -10n \log_{10}(d) + A$$

$$\Rightarrow d = 10^{\frac{A-RSSI}{10n}}$$

Trong đó:

- A : Cường độ tín hiệu tại khoảng cách 1m (Calibrated RSSI).
- n : Hệ số suy hao môi trường (Path-loss exponent, thường từ 2.0 - 4.0 trong nhà).

3.4. Giải thuật Đo ba cạnh (Trilateration)

Sau khi có 3 khoảng cách d_1, d_2, d_3 từ 3 vị trí cố định $(x_1, y_1), (x_2, y_2), (x_3, y_3)$, vị trí người dùng (x, y) được xác định bằng giao điểm của 3 đường tròn.

Hệ phương trình đường tròn:

$$(x - x_1)^2 + (y - y_1)^2 = d_1^2$$

$$(x - x_2)^2 + (y - y_2)^2 = d_2^2$$

$$(x - x_3)^2 + (y - y_3)^2 = d_3^2$$

Trong mã nguồn, Class IndoorTrilateration thực hiện giải hệ phương trình tuyến tính hóa từ các phương trình trên để trả về tọa độ $pos.x$ và $pos.y$.

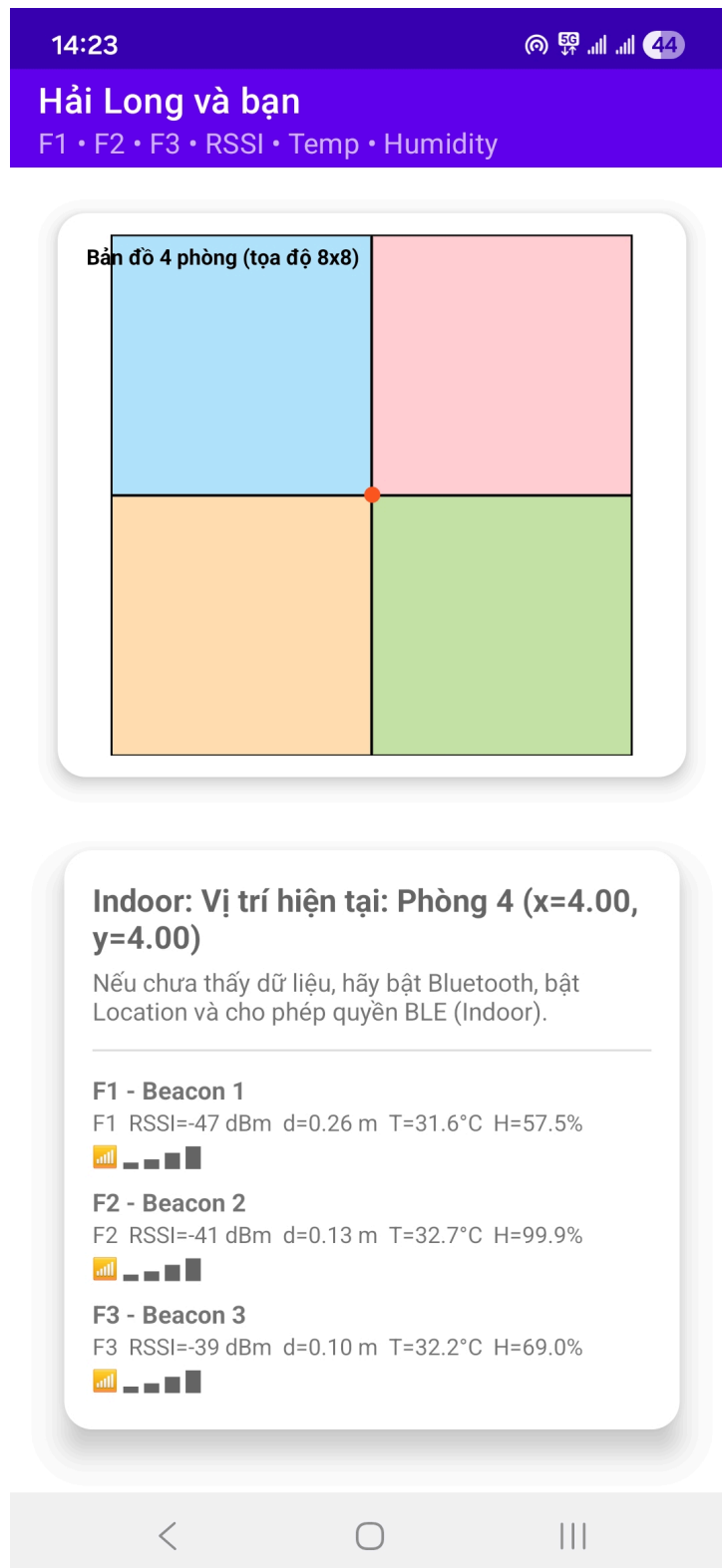
Sau đó, tọa độ này được so sánh với ranh giới các phòng (`room.contains(pos.x, pos.y)`) để xác định tên phòng hiện tại.

4. Kết quả thực nghiệm

4.1. Giao diện người dùng

Ứng dụng “Hải Long và bạn” hoạt động ổn định trên nền tảng Android. Giao diện trực quan bao gồm:

1. **Bản đồ nhiệt:** Hiển thị 4 vùng màu (Phòng 1-4) và chấm đỏ đại diện cho vị trí người dùng di chuyển theo thời gian thực.
2. **Dashboard:** Hiển thị chi tiết trạng thái của 3 Beacon: RSSI (kèm biểu tượng cột sóng), khoảng cách ước lượng (d), Nhiệt độ và Độ ẩm.



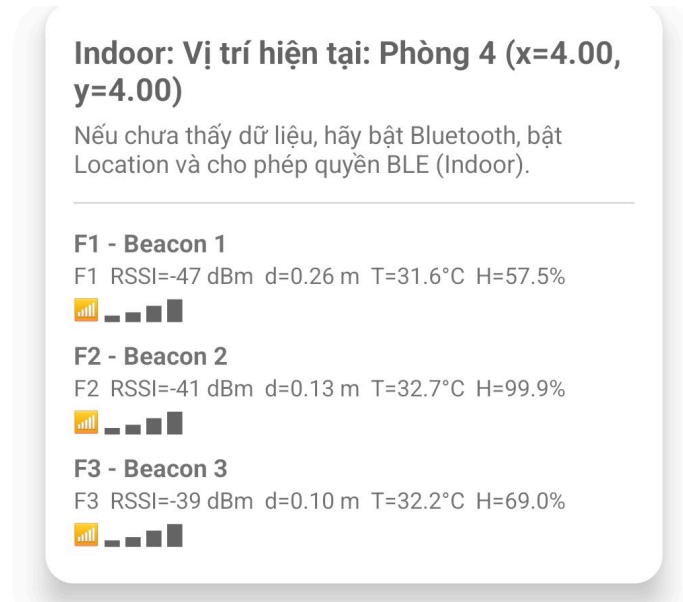
Hình 4: Giao diện định vị thời gian thực trên Android

4.2. Đánh giá độ chính xác

Qua video thực nghiệm, hệ thống đạt được các kết quả sau:

1. **Định vị:** Tọa độ (x, y) cập nhật liên tục (Ví dụ: $x = 4.04, y = 3.94$). Khi người dùng di chuyển, chấm đỏ trên bản đồ phản hồi mượt mà và chuyển đổi đúng tên phòng (Ví dụ: Từ “Phòng 3” sang “Phòng 4”).

2. **Dữ liệu môi trường:** Nhận đồng thời dữ liệu từ 3 cảm biến. Ví dụ:
- F1: 31.6°C, 57.5%
 - F2: 32.7°C, 99.9%
 - F3: 32.2°C, 69.0%
3. **Khoảng cách:** Các giá trị khoảng cách thay đổi hợp lý theo mức RSSI (RSSI -39dBm $\rightarrow d = 0.10m$; RSSI -47dBm $\rightarrow d = 0.26m$).



Hình 5: Dữ liệu chi tiết thu thập được từ các Anchor Nodes

5. Kết luận

Hệ thống định vị trong nhà (Indoor Positioning) đã được xây dựng thành công:

1. Ứng dụng Android xử lý tốt việc quét đa luồng (Multi-scanning) và giải mã dữ liệu thời gian thực.
2. Giải thuật Trilateration hoạt động hiệu quả, cho phép xác định vị trí tương đối của người dùng trong không gian 2 chiều.
3. Kết hợp thành công tính năng giám sát môi trường (IoT Monitor) vào hệ thống định vị (LBS).

Phụ lục: Mã nguồn quan trọng

Phần này trích dẫn các đoạn mã nguồn cốt lõi thể hiện các giải thuật xử lý chính của hệ thống.

A. Firmware EFR32 (C Language)

1. Cấu hình Timer và xử lý sự kiện đọc cảm biến (app.c)

```
1  #include "em_common.h"
2  #include "app_assert.h"
3  #include "sl_bluetooth.h"
4  #include "gatt_db.h"
5  #include "app_log.h"
6  #include "app_timer.h"
7
8  #include "sl_i2cspm_instances.h"
9  #include "aht20.h"
10 #include "lcd_display.h"
11
12 #include <string.h>
13 #include <stdio.h>
14 #include <stdlib.h>
15
16 #define DEVICE_NAME    "F3"
17 #define I2C_HANDLE      sl_i2cspm_i2c0
18
19 // ===== BIÊN TOÀN CỤC =====
20
21 static float current_temp = 0.0f;
22 static float current_hum  = 0.0f;
23
24 static app_timer_t sensor_timer;
25 static uint32_t    sensor_period_ms = 1000;    // 2s
26
27 static uint8_t     adv_handle = 0xFF;
28
29 // ===== HÀM PHỤ BCD =====
30
31 static uint8_t dec_to_bcd(uint8_t val)
32 {
33     return (uint8_t)((((val / 10) << 4) | (val % 10)));
34 }
35
36 // ===== CẬP NHẬT GÓI QUẢNG BÁ =====
37
38 static void update_ble_payload(float temp, float hum)
39 {
40     uint8_t adv_data[31];
41     uint8_t len = 0;
```

```
42
43 // ---- FLAGS ----
44 adv_data[len++] = 2; // length
45 adv_data[len++] = 0x01; // Flags
46 adv_data[len++] = 0x06; // General discoverable + BR/EDR not supported
47
48 // ---- TÊN THIẾT BỊ ----
49 const char *name = DEVICE_NAME;
50 uint8_t name_len = (uint8_t)strlen(name);
51 if (name_len > 29) name_len = 29;
52
53 adv_data[len++] = 1 + name_len; // length (type + name)
54 adv_data[len++] = 0x09; // Complete Local Name
55 memcpy(&adv_data[len], name, name_len);
56 len += name_len;
57
58 // ---- DỮ LIỆU CẢM BIẾN -> 4 BYTE BCD ----
59 int t_int = (int)temp;
60 int t_dec = (int)((temp - t_int) * 10.0f);
61 if (t_dec < 0) t_dec = -t_dec;
62
63 int h_int = (int)hum;
64 int h_dec = (int)((hum - h_int) * 10.0f);
65 if (h_dec < 0) h_dec = -h_dec;
66
67 uint8_t t_int_bcd = dec_to_bcd((uint8_t)t_int);
68 uint8_t t_dec_bcd = dec_to_bcd((uint8_t)t_dec);
69 uint8_t h_int_bcd = dec_to_bcd((uint8_t)h_int);
70 uint8_t h_dec_bcd = dec_to_bcd((uint8_t)h_dec);
71
72 // Manufacturer Specific Data
73 adv_data[len++] = 1 + 2 + 4; // length = type + 2 byte company ID + 4 dữ liệu
74 adv_data[len++] = 0xFF; // Manufacturer Specific
75 adv_data[len++] = 0xFF; // Company ID LSB (demo)
76 adv_data[len++] = 0x02; // Company ID MSB (demo)
77 adv_data[len++] = t_int_bcd;
78 adv_data[len++] = t_dec_bcd;
79 adv_data[len++] = h_int_bcd;
80 adv_data[len++] = h_dec_bcd;
81
82 // Gửi xuống stack
83 sl_bt_legacy_advertiser_set_data(
84     adv_handle,
85     sl_bt_advertiser_advertising_data_packet,
86     len,
87     adv_data);
```

```
88  }
89
90  // ===== TIMER ĐỌC CẢM BIẾN =====
91
92  static void sensor_timer_cb(app_timer_t *timer, void *data)
93  {
94      (void)timer;
95      (void)data;
96
97      if (aht20_read(I2C_HANDLE, &current_temp, &current_hum) == SL_STATUS_OK) {
98          // Cập nhật LCD
99          lcd_update(current_temp, current_hum, sensor_period_ms);
100
101          // Cập nhật BLE ADV
102          update_ble_payload(current_temp, current_hum);
103
104          // In UART không dùng %f
105          // Dùng 1 số lẻ thập phân: nhân 10
106          int16_t temp10 = (int16_t)(current_temp * 10.0f);
107          int16_t hum10  = (int16_t)(current_hum * 10.0f);
108
109          int t_int  = temp10 / 10;
110          int t_frac = abs(temp10 % 10);
111
112          int h_int  = hum10 / 10;
113          int h_frac = abs(hum10 % 10);
114
115          app_log("D:%d.%d,%d.%d\r\n",
116                t_int, t_frac,
117                h_int, h_frac);
118      } else {
119          app_log("AHT20 read FAIL\r\n");
120      }
121  }
122
123  // ===== BLUETOOTH EVENTS =====
124
125  void sl_bt_on_event(sl_bt_msg_t *evt)
126  {
127      sl_status_t sc;
128      uint32_t evt_id = SL_BT_MSG_ID(evt->header);
129
130      switch (evt_id) {
131          case sl_bt_evt_system_boot_id:
132              app_log("BLE boot\r\n");
133      }
```

```
134 // Tạo advertising set
135 sc = sl_bt_advertiser_create_set(&adv_handle);
136 app_assert_status(sc);
137
138 // Chu kỳ quảng bá ~50ms (80 * 0.625 ms)
139 sc = sl_bt_advertiser_set_timing(adv_handle,
140                                 80, 80,
141                                 0, 0);
142 app_assert_status(sc);
143
144 // Phát trên cả 3 kênh 37,38,39
145 sc = sl_bt_advertiser_set_channel_map(adv_handle, 7);
146 app_assert_status(sc);
147
148 // Payload ban đầu (giá trị ảo, sẽ được cập nhật sau)
149 update_ble_payload(25.3f, 64.7f);
150
151 // Bắt đầu quảng bá non-connectable
152 sc = sl_bt_legacy_advertiser_start(adv_handle,
153                                   sl_bt_advertiser_non_connectable);
154 app_assert_status(sc);
155
156 app_log("Started advertising\r\n");
157 break;
158
159 default:
160     break;
161 }
162 }
163
164 // ===== APP INIT & PROCESS =====
165
166 void app_init(void)
167 {
168     lcd_init_and_clear();
169
170     if (aht20_init(I2C_HANDLE) == SL_STATUS_OK) {
171         app_log("AHT20 OK\r\n");
172     } else {
173         app_log("AHT20 FAIL\r\n");
174     }
175
176     // Bắt đầu timer đọc cảm biến
177     app_timer_start(&sensor_timer,
178                   sensor_period_ms,
179                   sensor_timer_cb,
```

```
180         NULL,  
181         true);  
182     }  
183  
184     void app_process_action(void)  
185     {  
186         // Không cần xử lý gì thêm trong vòng lặp chính  
187     }
```

2. Driver I2C đọc AHT20 mức thanh ghi (aht20.c)

```
1  sl_status_t aht20_read(sl_i2cspm_t *i2cspm_handle, float *temp, float  
   *hum)  
2  {  
3      // Gửi lệnh Trigger Measurement: 0xAC, 0x33, 0x00  
4      uint8_t cmd_trigger[] = {0xAC, 0x33, 0x00};  
5      // ... (Code gửi lệnh I2C Write) ...  
6  
7      // QUAN TRỌNG: Chờ 80ms để cảm biến chuyển đổi ADC  
8      sl_sleeptimer_delay_millisecond(80);  
9  
10     // Đọc 7 byte dữ liệu và xử lý bit  
11     // ... (Code I2C Read) ...  
12  
13     uint32_t raw_hum = ((uint32_t)read_data[1] << 12) | ((uint32_t)read_data[2] <<  
   4) | ((uint32_t)read_data[3] >> 4);  
14     *hum = ((float)raw_hum / 0x100000) * 100.0f;  
15     return SL_STATUS_OK;  
16 }
```

B. PC Application (C/Win32 API)

Xử lý dữ liệu UART và lưu vào SQLite (TEST.c)

```
1  // Phân tích chuỗi nhận được từ UART: "D:temp,hum"  
2  if (sscanf(d_ptr, "D:%f,%f", &temp, &hum) == 2) {  
3      char time_buf[30];  
4      get_timestamp(time_buf);  
5  
6      // Thực thi câu lệnh SQL INSERT trực tiếp  
7      char sql[512];  
8      sprintf(sql, "INSERT INTO SensorLog (Timestamp, Temperature, Humidity)  
   VALUES ('%s', %.2f, %.2f);",  
9          time_buf, temp, hum);  
10     sqlite3_exec(db, sql, 0, 0, &err_msg);  
11 }
```

C. Android Application (Kotlin)

Mã nguồn xử lý chính trên Android, bao gồm quy trình xin quyền truy cập, quét BLE theo bộ lọc tên thiết bị và cập nhật giao diện người dùng thời gian thực.

1. Khởi tạo và Cấp quyền (IndoorMainActivity.kt)

Hàm checkPermissionsAndStart đảm bảo ứng dụng có đủ quyền truy cập Location và Bluetooth Scan trước khi hoạt động.

```
1 private fun checkPermissionsAndStart() { Kotlin
2     val neededPermissions = mutableListOf<String>()
3     if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
4         neededPermissions.add(Manifest.permission.BLUETOOTH_SCAN)
5         neededPermissions.add(Manifest.permission.BLUETOOTH_CONNECT)
6     } else {
7         neededPermissions.add(Manifest.permission.ACCESS_FINE_LOCATION)
8         neededPermissions.add(Manifest.permission.ACCESS_COARSE_LOCATION)
9     }
10    // ... (Code kiểm tra và request quyền) ...
11    if (notGranted.isNotEmpty()) {
12        ActivityCompat.requestPermissions(this, notGranted.toTypedArray(), 2002)
13    } else {
14        startBleScan()
15    }
16 }
```

2. Quét và Lọc thiết bị F1/F2/F3 (IndoorMainActivity.kt)

Callback onScanResult thực hiện lọc thiết bị theo tên và giải mã dữ liệu cảm biến từ Manufacturer Specific Data (0x02FF).

```
1 private val scanCallback = object : ScanCallback() { Kotlin
2     override fun onScanResult(callbackType: Int, result: ScanResult?) {
3         super.onScanResult(callbackType, result)
4         result ?: return
5         val scanRecord = result.scanRecord
6         val devName = scanRecord?.deviceName ?: result.device.name ?: return
7
8         // LỌC THEO TÊN F1/F2/F3
9         val beacon = beacons.find { it.bleName.equals(devName, ignoreCase = true) } ?: return
10
11        beacon.addRssi(result.rssi)
12
13        // Đọc Manufacturer Data: 0x02FF chứa T/H dạng BCD
14        val mfDataSparse = scanRecord?.manufacturerSpecificData
15        if (mfDataSparse != null && mfDataSparse.size() > 0) {
16            val data = mfDataSparse.get(0x02FF)
17            if (data != null && data.size >= 4) {
```

```
18         val tInt = bcdToInt(data[0])
19         val tDec = bcdToInt(data[1])
20         // ... (Xử lý độ ẩm tương tự) ...
21         beacon.temperature = tInt + tDec / 10f
22         beacon.humidity = hInt + hDec / 10f
23     }
24 }
25 updateUiAndPosition()
26 }
27 }
```

3. Cập nhật Vị trí và UI (IndoorMainActivity.kt)

Hàm updateUiAndPosition gọi thuật toán Trilateration và cập nhật giao diện người dùng (Dashboard & Map).

```
1 private fun updateUiAndPosition() { Kotlin
2     runOnUiThread {
3         updateBeaconText()
4         // Lấy 3 beacon
5         val b1 = beacons.getOrNull(0); val b2 = beacons.getOrNull(1); val b3 =
            beacons.getOrNull(2)
6         if (b1 == null || b2 == null || b3 == null) return@runOnUiThread
7
8         val d1 = b1.estimatedDistance(); val d2 = b2.estimatedDistance(); val d3 =
            b3.estimatedDistance()
9         if (d1 == null || d2 == null || d3 == null) return@runOnUiThread
10
11         val p1 = PointF(b1.posX, b1.posY)
12         val p2 = PointF(b2.posX, b2.posY)
13         val p3 = PointF(b3.posX, b3.posY)
14
15         // TÍNH TOÁN VỊ TRÍ (TRILATERATION)
16         val pos = IndoorTrilateration.calculatePosition(p1, d1, p2, d2, p3, d3)
17
18         if (pos != null) {
19             houseMapView.setUserPosition(pos.x, pos.y)
20             val room = rooms.find { it.contains(pos.x, pos.y) }
21
22             if (room != null) {
23                 tvCurrentRoom.text = "Indoor: Vị trí hiện tại: ${room.name}
                    (x=%.2f, y=%.2f)".format(pos.x, pos.y)
24             } else {
25                 tvCurrentRoom.text = "Indoor: Ngoài vùng (x=%.2f,
                    y=%.2f)".format(pos.x, pos.y)
26             }
27         }
28     }
```

